



Java程序设计

Ch03-1 面向对象基础

余 烨

本章学习提示

- 本章我们主要学习面向对象编程的基本概念。
  - 了解类与对象的概念及面向对象编程的思想
  - 访问控制符的使用
  - this、static 和 final 的使用
  - 内部类的概念

面向对象和面向过程的区别

- 举例：大象装进冰箱
- 面向过程
  - 打开冰箱、存储大象、关上冰箱
  - 面向过程的思想，强调的是过程（动作）
- 面向对象
  - 抽象类：冰箱、大象、人
  - 冰箱——属性：容量，开、关或工作的状态；
  - 人——动作：打开、关闭、放
  - 大象——属性：重量、体积

目　录

1 类 & 对象 &面向对象

2 对象的内存模型

3 this & static & final

4 访问控制符

类的定义

类定义的格式:

[类修饰符] **class** 类名 [extends 父类名] [implements 接口名]

{

成员变量;

构造方法;

成员方法;

}

People people = new People();

People people = new People(id, name);

创建类对象一般形式:

类名 对象变量 = new 类名(参数1，参数2，…);

或者

类名 对象变量;

对象变量 = new 类名(参数1，参数2，…);

Practice - 类的定义

- 定义一个People类，包括：
  - 两个成员变量：姓名，住址
  - 构造方法，用于设置两个成员属性的初值
  - 成员方法 setInfo，供外部调用，改变两个成员属性的值
  - 成员方法 showName，供外部调用，返回“姓名”的值
- 实例化一个People类的对象，并打印其姓名

Example4_01

```
class People
{
        String name;   //姓名
        String address;   //住址
        People()   {        }
        People(String strName, String strAdd)
        {
                name = strName;
                address = strAdd;
         }
         void setInfo(String strName, String strAdd)
         {
                 name = strName;
                 address = strAdd;
          }
          String showName()
          {
                  return name;
          }
```

```
class TestPrint
{
        public static void main(String[] args)
        {
                People p1 = new People("George", "New
York");
                System.out.println( p1.showName() );
         }
}
```

名词解释

- 成员变量
  - 成员变量是类的一个组成部分，一般是基本数据类型，也可以是引用类型
  - 如何访问（使用）对象的成员变量
    - 对象名.变量名
- 成员方法
  - == 函数
  - 作用：
    - 对该类内的变量进行各种操作；
    - 与其它类的对象进行信息交流，作为类与外部进行交互通信的接口

构造方法

- 构造方法是一个类方法中方法名与类名相同的类方法，它的作用是当使用new关键字创建一个对象时，自动调用并对新建对象进行初始化操作
- 语法格式：
  - [修饰符]构造方法（构造器）名称(形参列表)
  - { 初始化语句；}
- 说明：
  - 修饰符可以省略，也可以是public、protected、private其中之一
  - 构造方法名称必须和类名相同
  - 形参列表和定义方法形参列表的格式完全相同

构造方法的特点

- 构造方法没有返回值，不定义返回类型

  构造方法不能定义返回值类型声明，也不能使用void定义构造方法没有返回值。如果为构造方法定义返回值类型，或使用void定义构造方法没有返回值，编译时不会出错，但java会把这个所谓的构造方法当成成员方法来处理

- 构造方法可以重载，针对参数数量和类型定义多个同名构造方法
- 构造方法可以继承，子类可以继承父类的构造方法
- 构造方法不能由编程人员显式地直接调用，它是在对象创建时由new运算符自动调用的
- 构造方法只用来对类实例进行初始化，目的在于简化初始化操作

构造方法

- **每个类都至少有一个构造方法**，类中可以不定义构造方法，如果程序员没有为类定义构造方法，系统会自动为该类生成一个默认的构造方法。
  - 默认构造方法的名字与类名相同，它的参数列表和方法体均为空，自动将所有实例变量初始化为**默认值**(数值型为0，布尔型为false，字符型为‘\u0000’(‘a’)，对象型为null).
- 但是一旦定义了自己的构造方法，默认构造方法将不再起作用.

利用javap查看类里的方法：javap 类名

OOP语言

- OOP的**基本思想**是把人们对现实世界的认识过程应用到程序设计中。把现实世界的事物抽象成对象，它有两个特征：**状态**（state）和**行为**（behavior）
- 软件的对象概念是由现实世界对象抽象而来。把现实世界对象的**状态保存在软件对象的变量**中，现实世界对象的**行为通过软件对象的方法（函数）来实现**。

类与对象的概念

- 把相似的对象划归成一个类。
- 类是对象的“模板”，对象是类的实例.
- 类与对象之间的关系可以看成是抽象与具体的关系。其中类是抽象的，对象是具体的
  - 类描述了对象的属性和行为
  - 对象是类的一个实例
- 比如，当设计灯泡的时候要求每一个灯泡都必须具有型号这个属性，同时还要能够进行打开、关闭等操作
  - 因此当把灯泡这些共同的属性和行为集中起来的时候就抽象出一个灯泡类
  - 每一个具有这些属性和行为的，具体的灯泡就是这个灯泡类的对象或者称为实例

面向对象程序设计方法

- 面向对象程序设计方法是：

  对象 = （算法 + 数据结构）

  程序 = （对象 + 对象 + ...）

- 算法与数据结构是一个整体，算法离不开数据结构，算法只能适用于特定的数据结构。
- 面向对象中，算法与数据结构被捆绑成一个类
- **设计方法**：先设计类；然后根据类创建对象

Actor类

属性：

String name; //姓名

float height; //身高

String company； //属于哪个公司

方法：

public Actor(); //无参构造方法

//有参构造方法

pulbic Actor(String name, float height);

//设置演员信息

void setInfo(String name, float height);

问题：

创建一个明星对象



Yangzi
1.67

最有名的明星

修改

Dongyu
1.62

A工作室

演员A



请问修改之后会出现什么问题？
对演员A有什么影响？

目　录

1 类 & 对象 & 面向对象

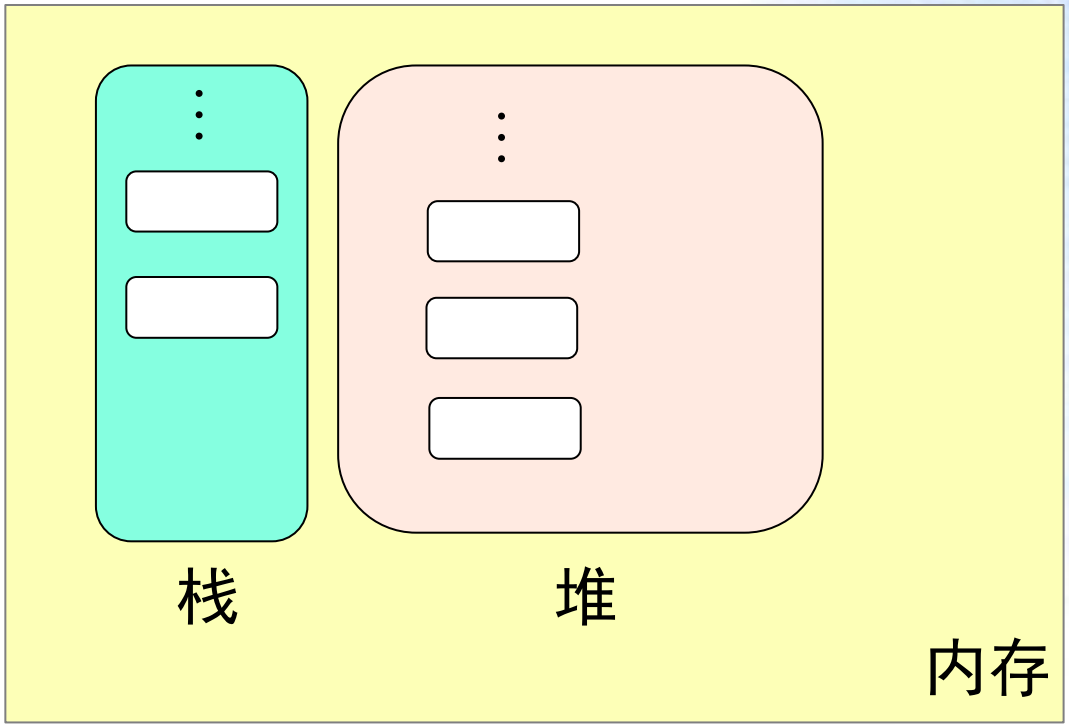
2 对象的内存模型

3 this & static & final

4 访问控制符

数据都存在哪里？

- 寄存器、栈、堆、静态存储、常量存储和非内存的存储
- 栈
  - 存放一些标识符、方法中定义的局部变量、参数等；
  - 存在栈中的数据大小和生存期必须是确定的
  - 存取速度比堆快
- 堆
  - 类实例和数组对象分配内存的区域
  - 可以动态地分配内存大小
  - 存储速度较慢





AndroidDoll

属性：
String color; //颜色
int id; //标识号

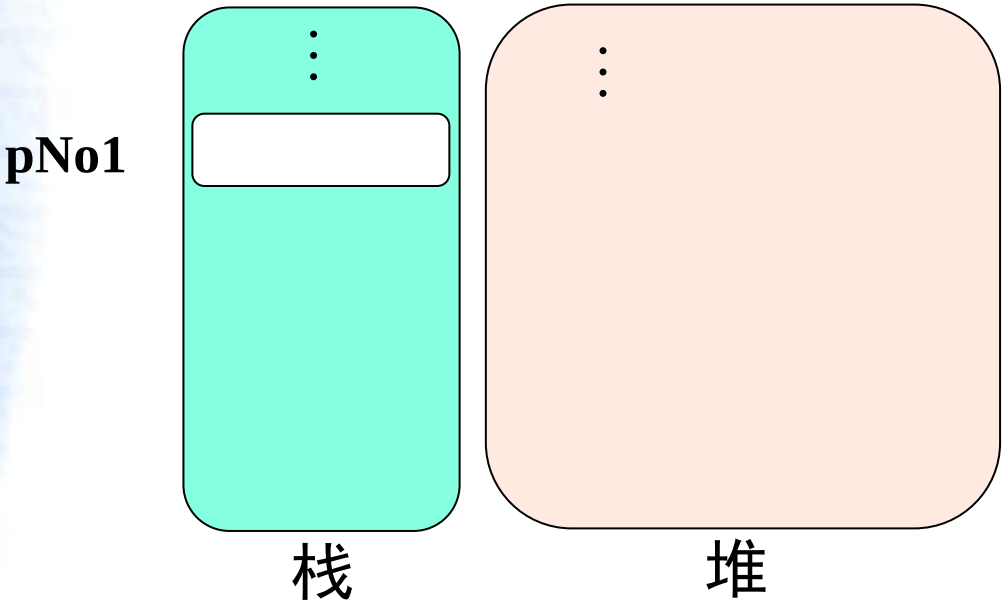
方法：
void setInfo(String color, int id); // 设置或更改信息

分 析——定义

- 用AndroidDoll声明一个对象变量（局部变量）pNo1：

  AndroidDoll pNo1;

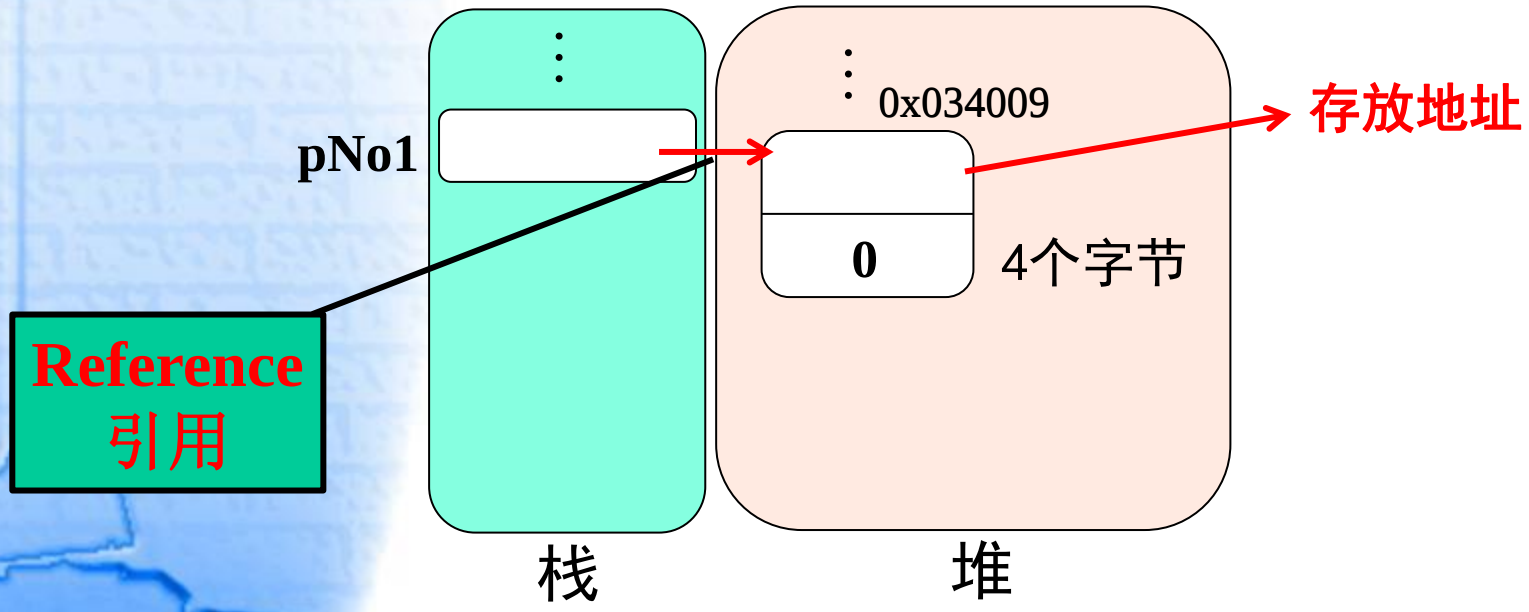
- 内存模型
  - 在栈中为pNo1对象分配一块内存空间，用来存储一个内存地址
  - 尚未为pNo1对象真正分配内存空间



分 析——初始化

- 对变量进行初始化
  - pNo1 = new AndroidDoll();
- 内存模型
  - 在堆中为“pNo1”对象分配一块内存空间，内存的大小视其成员变量的情况而定。

String是对象类型



分 析——赋值

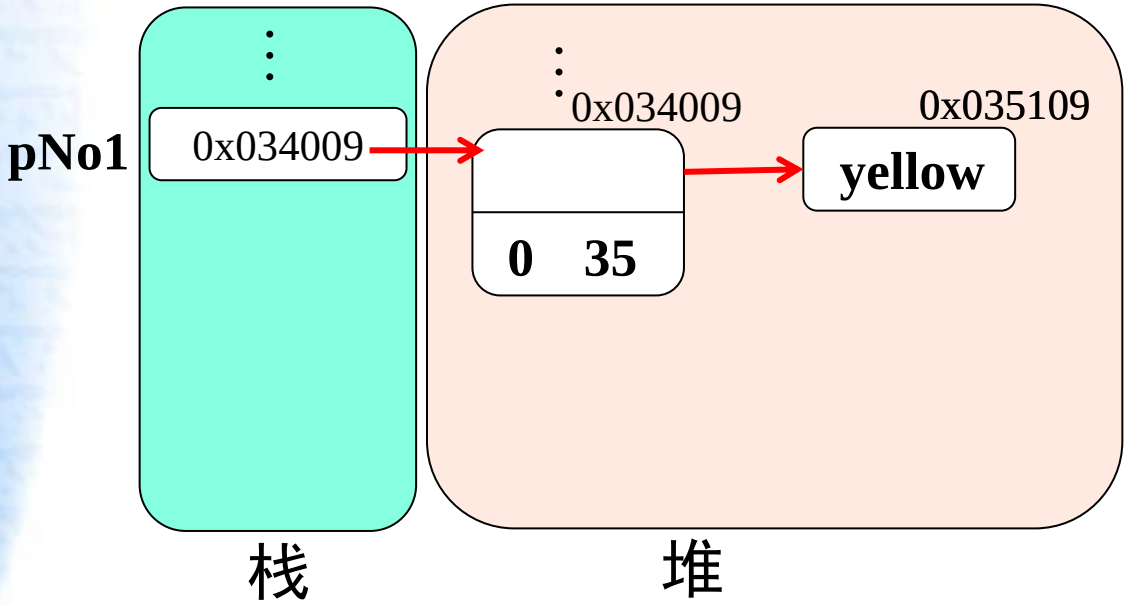


- 对对象进行赋值

  pNo1.setInfo(“yellow”, 35);

- 内存模型

  - 对于基本数据类型的成员变量，将值存储在内存中；对于对象类型的成员变量，需要再次分配空间存储其值



分 析——对象数组

- 定义一个AndroidDolll类型的数组

```
AndroidDoll pDoll[];
```

- 初始化数组的大小为2；

```
pDoll = new AndroidDoll[2];
```

- 为数组中的每个对象初始化

```
pDoll[0] = new AndroidDoll();
pDoll[1] = new AndroidDoll();
```

- 为每个对象赋值

```
pDoll[0].setInfo("green", 24);
pDoll[0].setInfo("red", 30);
```

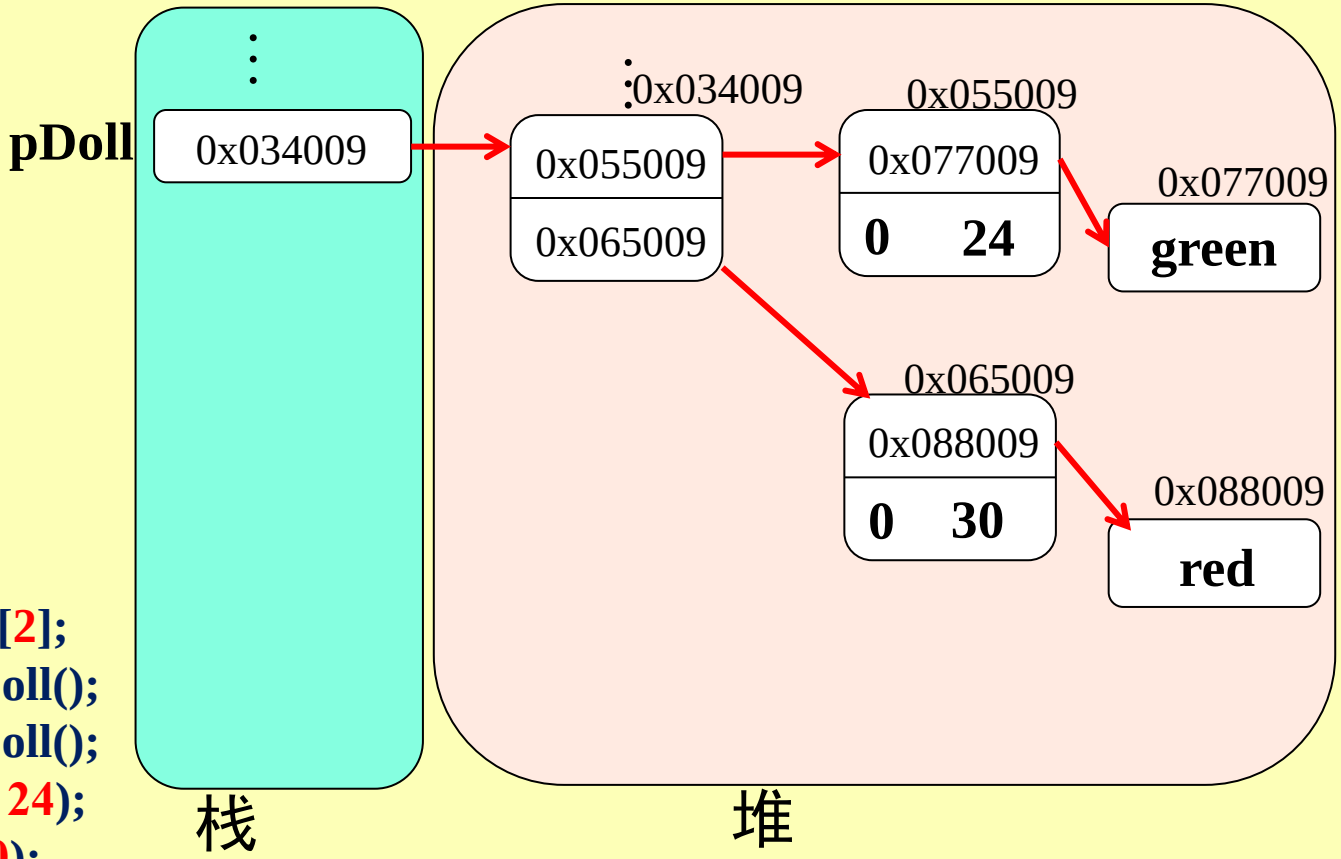


数组是对象类型



分 析——对象数组

- 数组定义
- 数组初始化
- 数组中每个元素初始化
- 赋值




```
AndroidDoll pDoll[];
pDoll = new AndroidDoll[2];
pDoll[0] = new AndroidDoll();
pDoll[1] = new AndroidDoll();
pDoll[0].setInfo("green", 24);
pDoll[0].setInfo("red", 30);
```

小 结

- 对于对象类型的局部变量，仅在栈内存中存储对象所在的内存地址（memory address）（这个内存地址被称为是对对象的引用reference），而真正的对象则存储在堆内存中。
- 对于对象类型的成员变量，需要在堆内存中开辟两块存储区域，一种用来存放变量的内存地址，一块用来存放赋值后变量的值。

问题：

创建一个明星对象



最有名的明星

Yangzi
1.67

A工作室

演员A

修改

Dongyu
1.62

属性：
String name; //姓名
float height; //身高

行为：
setInfo(String name, float height);

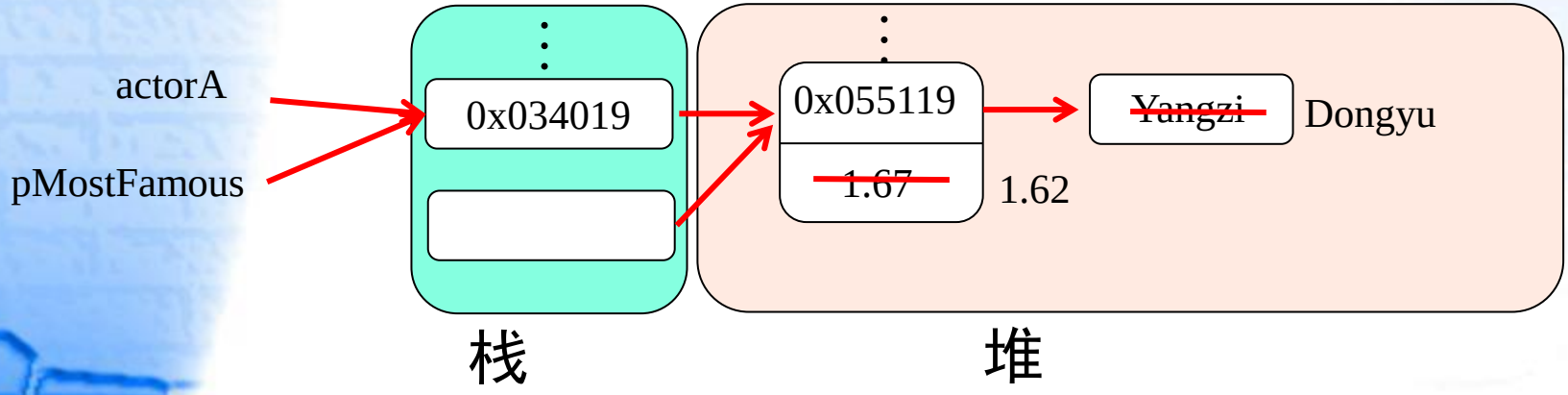


请问修改之后会出现什么问题？对演员A有什么影响？

```
Actor actorA;      //创建演员A
actorA = new Actor();     //初始化
actorA.setInfo("Yangzi", 1.67);    //设置对象信息

Actor pMostFamous;          //创建pMostFamous对象
pMostFamous = actorA;       //赋值

pMostFamous.setInfo("Dongyu", 1.62); //修改pMostFamous球员信息
//查看上述信息的修改是否影响到演员A
System.out.prinln(actorA.name);
System.out.println(actorA.height);
```



目　录

1 类 & 对象 & 面向对象

2 对象的内存模型

3 this & static & final

4 访问控制符

特殊变量this

- 当类中有两个同名变量，一个属于类（类的成员变量），而另一个属于某个特定的方法（方法中的局部变量），使用this区分成员变量和局部变量。
- 使用this简化构造函数的调用。

Example4_02

示 例

```
public class Apple
{
    public String name, color, weight;
    //无参构造方法
    public Apple() { }
    //两个参数的构造方法, 对name、color赋值
    public Apple(String name, String color)
    {   this.name = name;
        this.color = color;   }
    //三个参数的构造方法
    public Apple(String name, String color, float weight)
    {
        this(name, color);
        this.weight = weight;   }
}
```

通过this调用两个参数的构造方法

static

- static 修饰的成员变量为静态变量
- static 修饰的方法为静态方法

- ☑ 静态变量和静态方法属于某个类，而不属于类的对象
- ☑ ***静态变量***保存在内存区的公共存储单元中，而不是保存在某个对象的内存区中。任何一个类对象在访问它时都会得到相同的数值。
- ☑ 如果某一个类对象对它进行了修改，所有对象的此成员变量也都做了修改。
- ☑ 静态变量和静态方法不用进行实例化操作，使用时***方法前加类名而不是对象名，*** 即： ***类名.方法***。
- ☑ 在静态方法中不能调用非静态的方法和引用非静态的成员变量；反之，则可以。

```
// 定义一个学生档案类，包括学生姓名、性别和所在学校
// 使用static修饰成员变量
class Student {
    String name;
    String  sex;
    static String school="合肥工业大学"; }
public class ch05_06_Student {
    public static void main(String args[]) {
        Student me=new Student();
        Student you=new Student();
        me.name="李明 ";
        me.sex="男 ";
        you.name="张敏 ";
        you.sex="女";
        //对类成员变量赋值
        Student.school="安徽大学";
        System.out.println(me.name+" "+me.sex+" "+me.school);
        System.out.println(you.name+" "+you.sex+"
"+you.school);
    } }
```

Example4_03

final

- final 修饰的变量为最终成员变量（=常量）
- final 修饰的方法为最终方法

- ☑ 使用final关键字定义常量，如：

  final double PI = 3.1415926;
- ☑ 作为一种约定，在定义常量时，通常采用大写形式；为了节省内存，通常将常量声明为static
- ☑ **final修饰的成员变量一般是在声明时进行初始化，一旦最终成员变量被赋值，则它在整个程序执行过程中不能改变。**
- ☑ **final修饰的方法不能被子类重载**
- ☑ **final修饰的类不能被继承**

final

- 在类的成员变量中定义：

  final double PI;

  // error; 因为用final声明的常量，必须在声明时初始化

- 但是在main里面，声明：

  final double PI;

  如果Pi未被使用，则不会出错，一旦Pi被使用，就会报错，Pi没有被初始化

Code - final成员变量

```
//定义类及其方法，引用此方法时
//通过final实参对类进行初始化操作
class Calculate {
    private double r;
    final double PI=3.14159;
    void area(double r) {
      double x;
      x=PI*r*r;
      System.out.println("Area=“ + x);
    }
}
public class ch05_07_Calculate {
    public static void main(String args[]) {
      Calculate   dd = new Calculate();
      dd.area(12.3);
    }   }
```

Example4_04

目　录

1 类 & 对象 & 面向对象

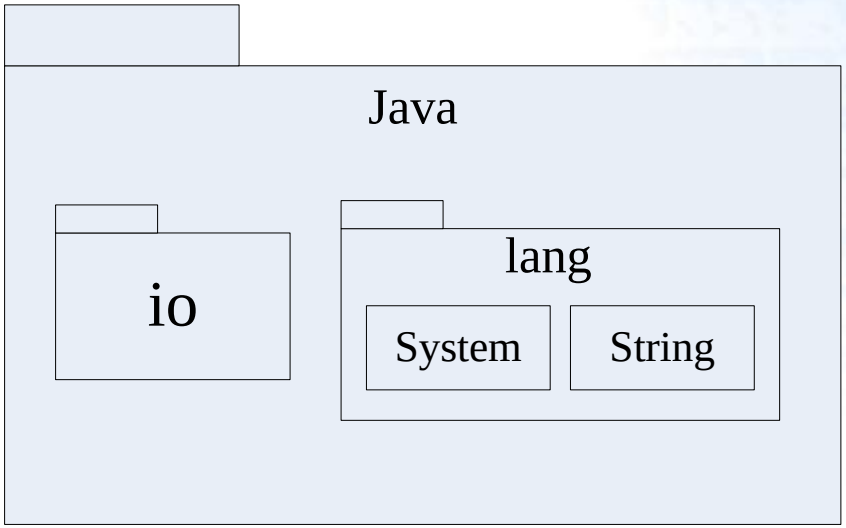
2 对象的内存模型

3 this & static & final

4 访问控制符

包（package）

- 为了便于管理大型软件系统中数目众多的类，解决类命名冲突的问题，Java引入了包（package）
- 包对应着文件系统的目录层次结构
- package语句必须是文件中的第一条语句，也就是说，在package语句之前，除了空白和注释之外不能有任何语句
- 在package语句中，用”.”来指明包（目录）的层次
  - package ch01
  - package cn.ch01



包（package）

- javac –d <中间文件所存放的路径><被编译程序名.java>
  - 如：javac –d . Test.java；javac –d D: Test.java
- 执行：Java package.类名 或者 java package/类名
- 被编译程序的第一行用package 包名；给出它被打入的包，该包与系统创建的子目录同名
- 例子：

```
package cn.shanghai;
class TestPackage {
    public static void main(String[] args)   {
    cn. shanghai.TestPackage t = new cn. shanghai.TestPackage();
    //或TestPackage t = new TestPackage();
   System.out.println("TestPackage");
              }
}
```

import语句

- import <包中的类型>
  - import util.*;

    引入util包中所有的类
  - import java.util.Arrays;

    引入java.util包中的Arrays类
- 在同一个包中的类可以相互引用，无需import语句
- Java.lang包
  - 提供利用 Java 编程语言进行程序设计的基础类
  - 在使用时不需导入

类的访问控制符

- 访问控制符：允许库创建者声明哪些东西是客户程序员可以使用的，哪些是不可使用的。这种访问控制的级别在“最大访问”和“最小访问”的范围之间
- 类的访问控制符号（说明符）
  - public
  - default（不加访问控制符时）
- 类的其它修饰符号
  - final
  - abstract

例子

- Test1.java文件:
  - package ch03;
  - class Test1 { } ⟶ default

- Test2.java文件:
  - package ch03; ⟶ 修改为：package ch05;
  - class Test2 import ch04.Test1;
  - { 测试不同包中Test1为public和default类型时的不同访问问题
  - public static void main(String[] args){
  - Test1 t1 = new Test1(); //同一个包中的两个类,可以互相访问
  - }}

结论1

- 假如类或者类成员*不含有*一个明确的访问说明，说明它具有*默认的访问控制特性（default）*。
- 默认的访问控制权规定类或者类成员*只能被同一个包中的其他代码所访问，在包外不可见*。这种访问特性称为**包访问性**。

结论2

- 无论是类或者类成员，只要被声明成**public**，就意味着**它可以被从任何地方访问**.
  - 一个类作为整体对程序的其他部分可见，并不能代表类内的所有属性和方法也同时对程序的其他部分可见，前者只是后者的必要条件
  - 类的属性和方法能否为所有其他类所访问，还要看这些属性和方法自己的访问控制符.
- 一个JAVA程序文件只能定义一个public类，而且程序文件必须与它同名。

结论3

- **final** + 类
  - 表明这个类是最终的类，不能由此类派生出其它的子类
- 例如：
  - Test2.java文件:
    - package ch04;
    - public **final** class Test2 { }
  - Test1.java文件:
    - ……
    - class Test1 **extends** Test2 {……}

      //观察出现的错误
- Java.lang包中的String类就是一个final类

方法的访问控制符

- 方法的访问控制符（说明符）
  - public
  - protected
  - default（不加访问控制符时）
  - private

方法的访问控制符

	public	protected	default	private
同类	✓	✓	✓	✓
同包不同类	✓	✓	✓	×
不同包，子类	✓	✓	×	×
不同包，非子类	✓	×	×	×

方法的访问控制符

	public	protected	default	private
同类	√	√	√	√
同包不同类	√	√	√	×
不同包，子类	√	√	×	×
不同包，非子类	√	×	×	×

总结

- 用private修饰的类成员只能被该类中的其他成员所访问，在该类外不可见
- 用default修饰的类成员在同包中可见
- 用protected修饰的类成员不仅在同包中可见，而且对于其他包中该类的子类也是可见的(protected可以用于继承情况中)
- 在任何地方都可以访问： public

本章总结

- 本章我们学习了面向对象编程的基本概念。
  - 类、对象 & 面向对象程序设计思想
  - 访问控制符的使用
  - this、static 和 final 的使用
  - 内部类的概念

Class is Dismissed!

